

Cypress

Interactive Web testing framework for E2E and Component Testing in JavaScript/TypeScript ecosystems.

What it is and when to use it

Cypress combines runner, assertions, debugging and an interactive app, fitting JS/TS teams that value fast feedback.

Official site and documentation

Use the official Cypress source as the primary reference: <https://www.cypress.io/>. Official documentation should be the first source for installation, supported versions, syntax changes and new features.

Prerequisites

Install a Node.js and package-manager version supported by your Cypress release and verify node/npm versions.

Step-by-step installation

Run `npm install cypress --save-dev`, then `npx cypress open`. The Launchpad configures E2E or Component Testing.

Initial configuration

Review `cypress.config`, `baseUrl`, `retries`, `viewport`, `timeouts` and `reporters`. Keep secrets in environment variables.

First test

Create a spec, use `cy.visit`, resilient selectors and assertions. Use `cypress open` while developing and `cypress run` in automation.

Selectors and commands

Prefer dedicated data attributes when appropriate and keep custom commands understandable.

Network and API

Use `cy.intercept` to observe, wait for or stub traffic; balance mocks with real integration coverage.

CI/CD and evidence

Run `cypress run` and preserve screenshots, videos and reports; validate Linux dependencies or use suitable official images.

Troubleshooting

Check Node version, binary download, proxies, permissions, cache and Linux dependencies.

How to start safely

Start from the official Cypress site and documentation. Confirm prerequisites, supported versions, operating system and required permissions. Learn in a controlled environment before using it in a critical workflow.

First QA-oriented use

Pick one small, verifiable task in Cypress, configure only what is necessary, run a simple example and confirm the outcome. Repeat with different data or conditions before adding abstractions.

Using it in a real project

Define where Cypress fits, who owns configuration, how credentials and data are handled and how results are shared. Integrate it into the team workflow instead of creating an isolated activity.

Adoption good practices

Standardize versions and configuration, document important decisions, keep secrets out of code and preserve reproducible examples. Reassess whether Cypress still delivers value as the product evolves.

Common issues and troubleshooting

When Cypress fails, verify versions, dependencies, permissions, connectivity, data and environment before assuming a product defect. Preserve the original error and reduce the issue to the smallest reproducible case.